



机器学习导论

第十四讲：强化学习 Value-Based Reinforcement Learning

信息学院

人工智能系

晁飞 副教授

fchao@xmu.edu.cn



目录

- Action-Value Functions
- Q-Learning (DQN)
- TD and TD 4 DQN

Action-Value Functions

Discounted Return

Definition: Discounted return (aka cumulative discounted future reward).

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$



- The return depends on actions $A_t, A_{t+1}, A_{t+2}, \dots$ and states $S_t, S_{t+1}, S_{t+2}, \dots$
- Actions are random: $P[A = a | S = s] = \pi(a|s)$. (Policy function.)
- States are random: $P[S' = s' | S = s, A = a] = p(s'|s, a)$. (State transition.)

Action-Value Functions $Q(s, a)$

Definition: Discounted return (aka cumulative discounted future reward).

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

Definition: Action-value function for policy π .

- $Q_\pi(s_t, a_t) = \mathbb{D}[U_t | S_t = s_t, A_t = a_t]$.



- Taken w.r.t. actions $A_{t+1}, A_{t+2}, A_{t+3}, \dots$ and states $S_{t+1}, S_{t+2}, S_{t+3}, \dots$
- Integrate out everything except for the observations: $A_t = a_t$ and $S_t = s_t$.

Action-Value Functions $Q(s, a)$

Definition: Discounted return (aka cumulative discounted future reward).

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

Definition: Action-value function for policy π .

- $Q_\pi(s_t, a_t) = \mathbb{D}[U_t | S_t = s_t, A_t = a_t]$.

Definition: Optimal action-value function.

- $Q^*(s_t, a_t) = \max_{\pi} Q_\pi(s_t, a_t)$.
- Whatever policy function π is used, the result of taking a_t at state s_t cannot be better than $Q^*(s_t, a_t)$.

Deep Q-Network (DQN)

Approximate the Q Function

Goal: Win the game ($\approx$ maximize the total reward.)

Question: If we know $Q^*(s, a)$, what is the best action?

Approximate the Q Function

Goal: Win the game ($\approx$ maximize the total reward.)

Question: If we know $Q^*(s, a)$, what is the best action?

- Obviously, the best action is $a^* = \operatorname{argmax}_a Q^*(s, a)$.

Q^* is an indicator of how good it is for an agent to pick action a while being in state s .

Approximate the Q Function

Goal: Win the game ($\approx$ maximize the total reward.)

Question: If we know $Q^*(s, a)$, what is the best action?

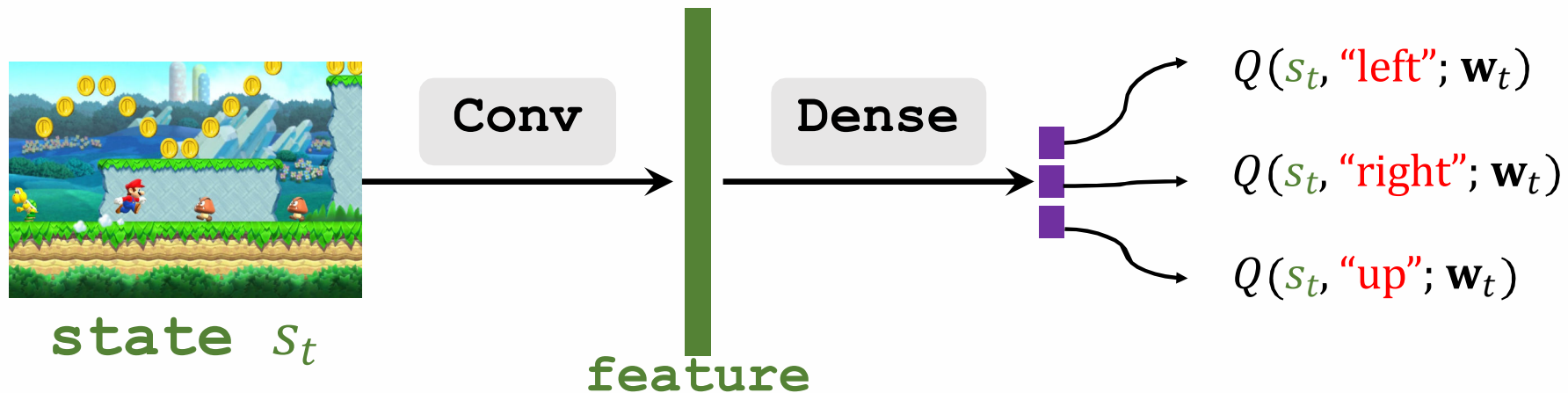
- Obviously, the best action is $a^* = \underset{a}{\operatorname{argmax}} Q^*(s, a)$.

Challenge: We do not know $Q^*(s, a)$.

- Solution: Deep Q Network (DQN)
- Use neural network $Q(s, a; \mathbf{w})$ to approximate $Q^*(s, a)$.

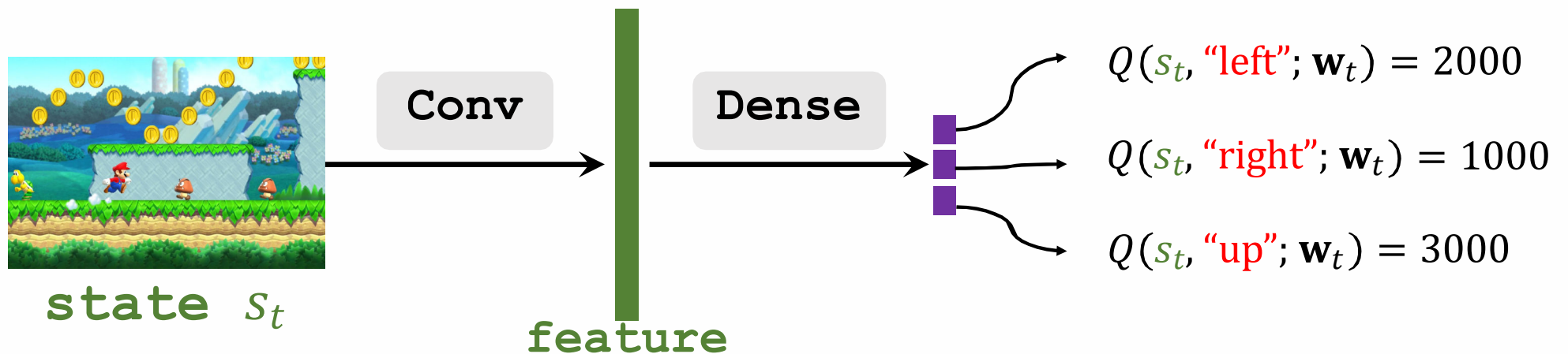
Deep Q Network (DQN)

- Input shape: size of the screenshot.
- Output shape: dimension of action space.



Deep Q Network (DQN)

- Input shape: size of the screenshot.
- Output shape: dimension of action space.



Question: Based on the predictions, what should be the **action**?

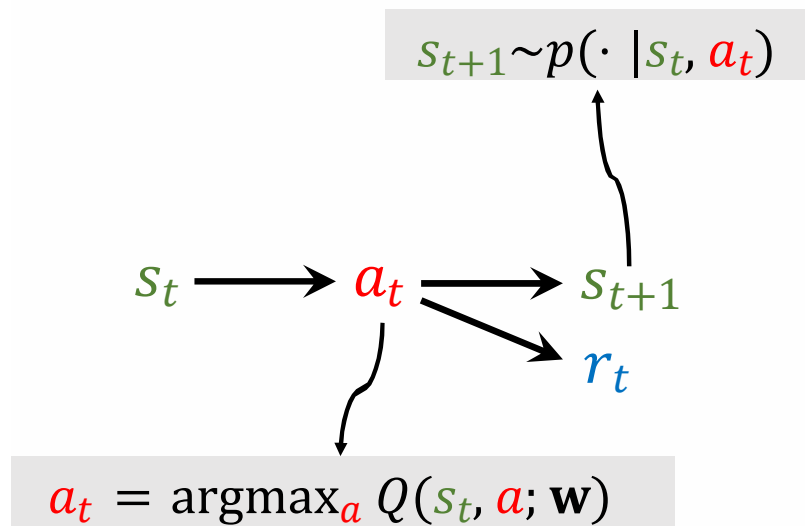
Apply DQN to Play Game

$s_t \longrightarrow a_t$

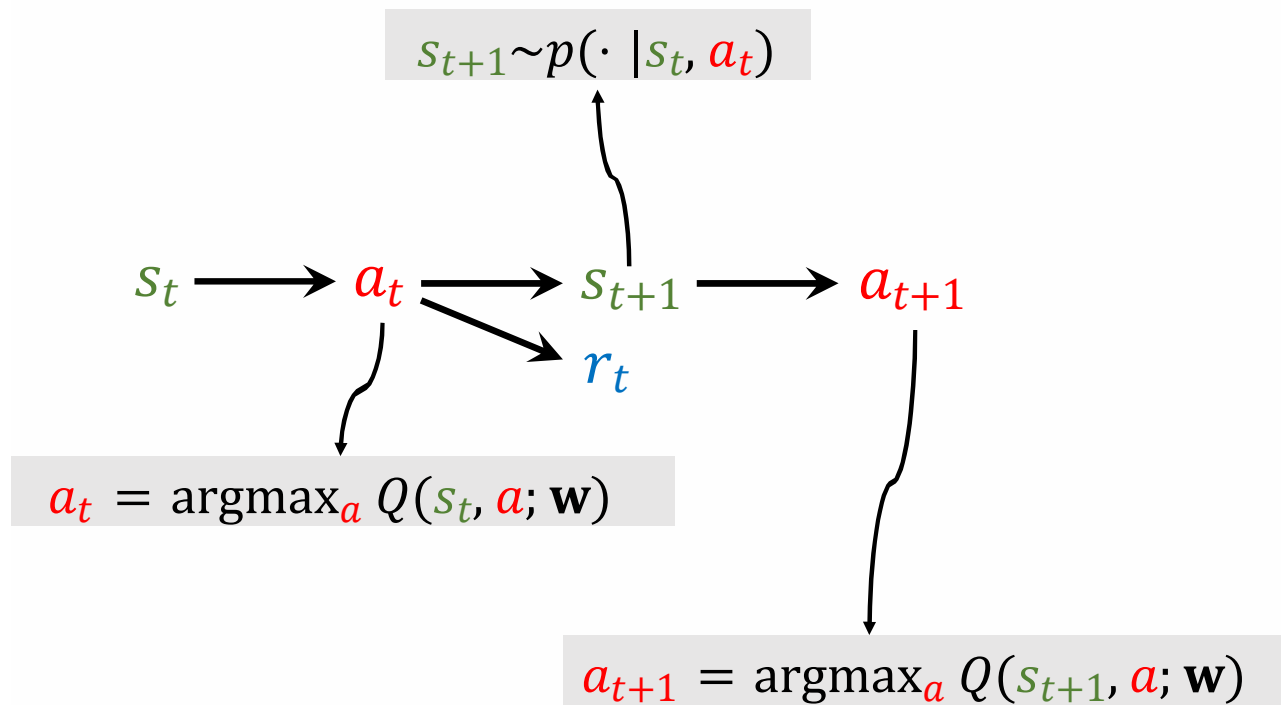


$$a_t = \operatorname{argmax}_a Q(s_t, a; \mathbf{w})$$

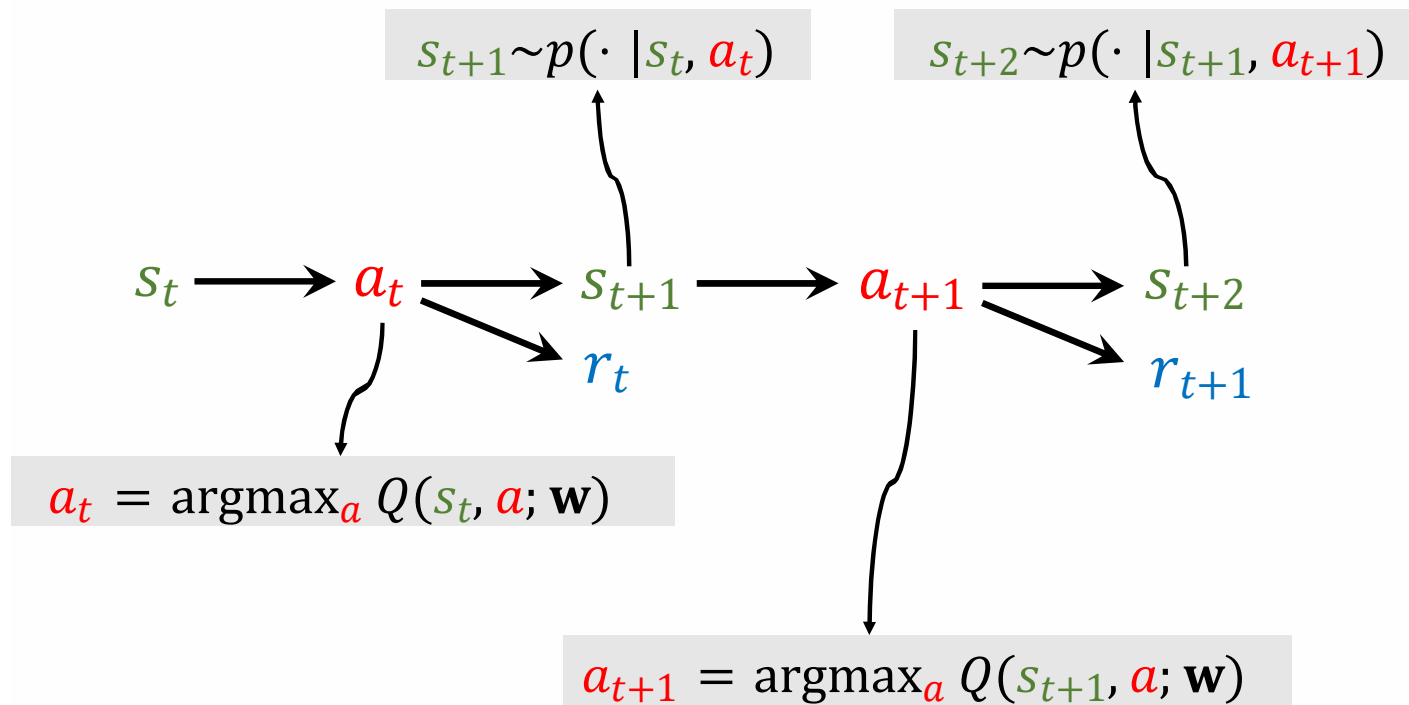
Apply DQN to Play Game



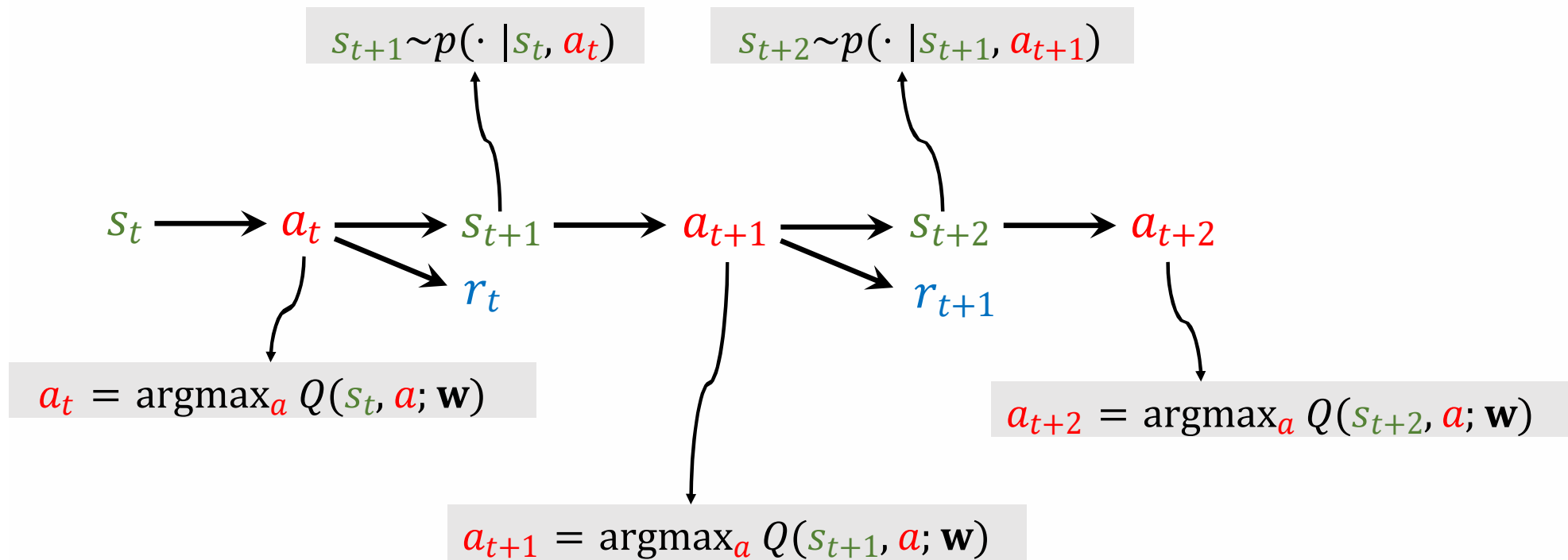
Apply DQN to Play Game



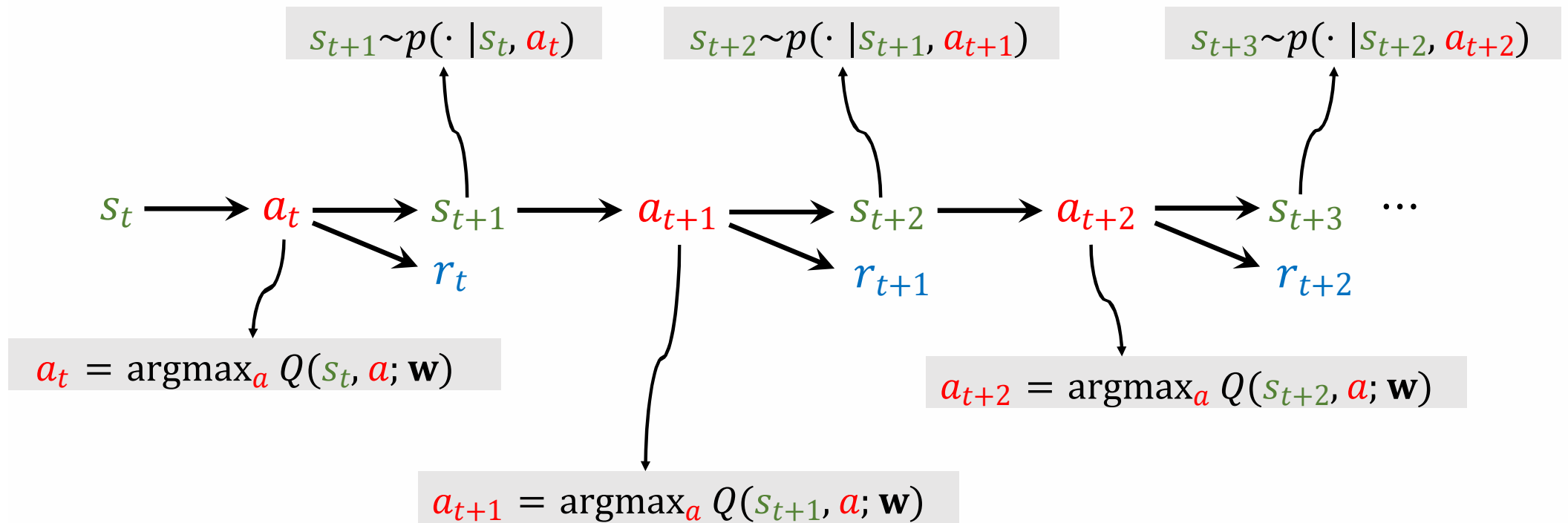
Apply DQN to Play Game



Apply DQN to Play Game



Apply DQN to Play Game



远古的方法：Q Table

□ 本质是LUT：

- 通过遍历得到这个表格
- 使用的时候根据已经遍历的状态查找动作

□ 也可以改造适用于基于策略的强化学习方法

	Action a_1	Action a_2	Action a_3	Action a_4	...
State s_1					
State s_2					
State s_3					
⋮					

Temporal Difference (TD) Learning

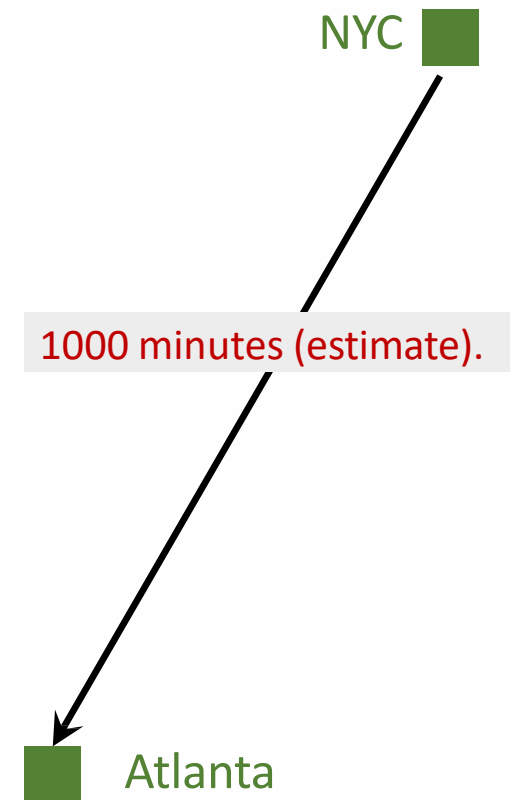
Reference

1. Sutton and others: [A convergent \$O\(n\)\$ algorithm for off-policy temporal-difference learning with linear function approximation](#). In *NIPS*, 2008.
2. Sutton and others: [Fast gradient-descent methods for temporal-difference learning with linear function approximation](#). In *ICML*, 2009.

Example

- I want to drive from NYC to Atlanta.
- Model $Q(\mathbf{w})$ estimates the time cost, e.g., 1000 minutes.

Question: How do I update the model?

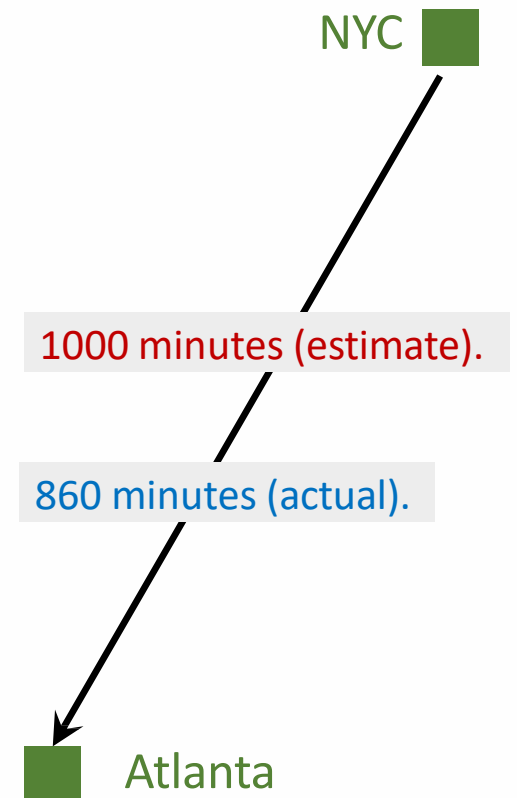


Example

- I want to drive from NYC to Atlanta.
- Model $Q(\mathbf{w})$ estimates the time cost, e.g., 1000 minutes.

Question: How do I update the model?

- Make a prediction: $q = Q(\mathbf{w})$, e.g., $q = 1000$.
- Finish the trip and get the target y , e.g., $y = 860$.

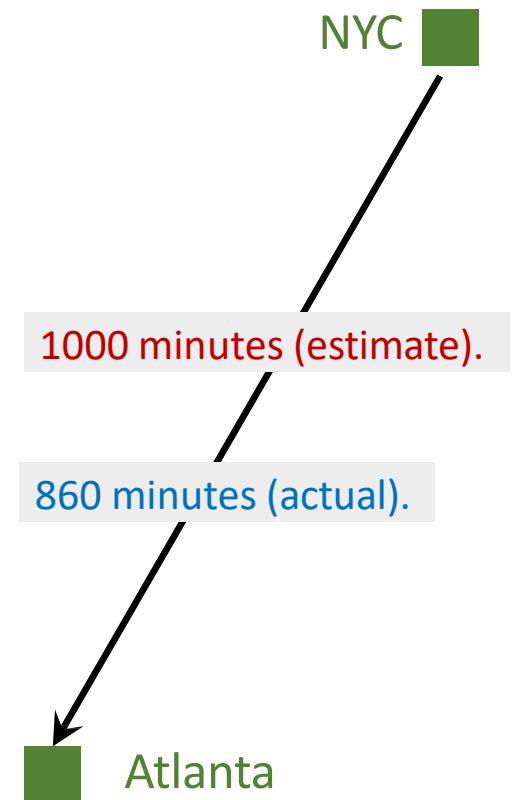


Example

- I want to drive from NYC to Atlanta.
- Model $Q(\mathbf{w})$ estimates the time cost, e.g., 1000 minutes.

Question: How do I update the model?

- Make a prediction: $q = Q(\mathbf{w})$, e.g., $q = 1000$.
- Finish the trip and get the target y , e.g., $y = 860$.
- Loss: $L = \frac{1}{2}(q - y)^2$.
- Gradient: $\frac{\partial L}{\partial \mathbf{w}} = \frac{\partial q}{\partial \mathbf{w}} \cdot \frac{\partial L}{\partial q} = (q - y) \cdot \frac{\partial a(\mathbf{w})}{\partial \mathbf{w}}$.
- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{\partial L}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.

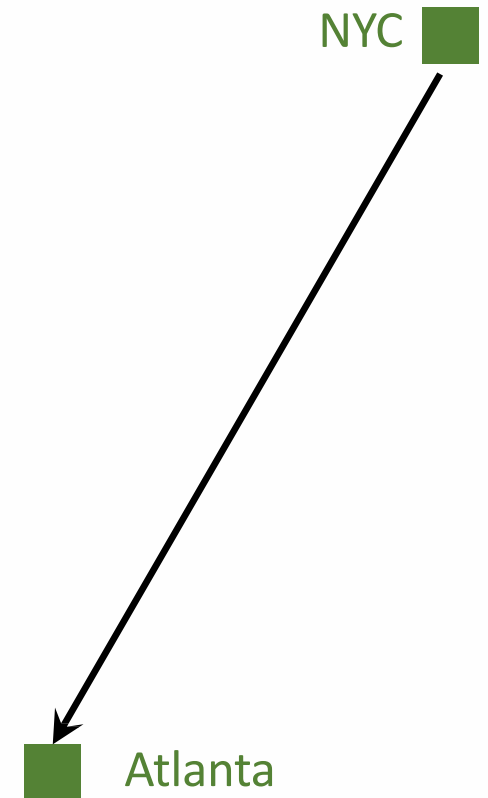


Example

- I want to drive from NYC to Atlanta.
- Model $Q(\mathbf{w})$ estimates the time cost, e.g., 1000 minutes.

Question: How do I update the model?

- Can I update the model **before finishing the trip**?

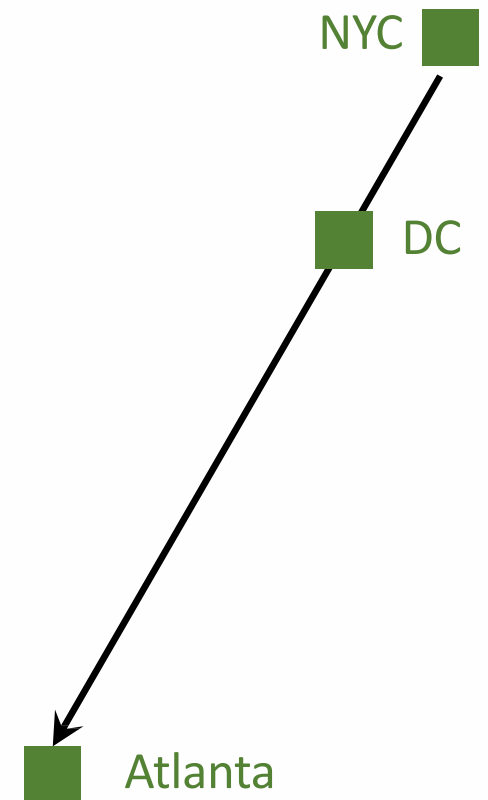


Example

- I want to drive from NYC to Atlanta (via DC).
- Model $Q(\mathbf{w})$ estimates the time cost, e.g., 1000 minutes.

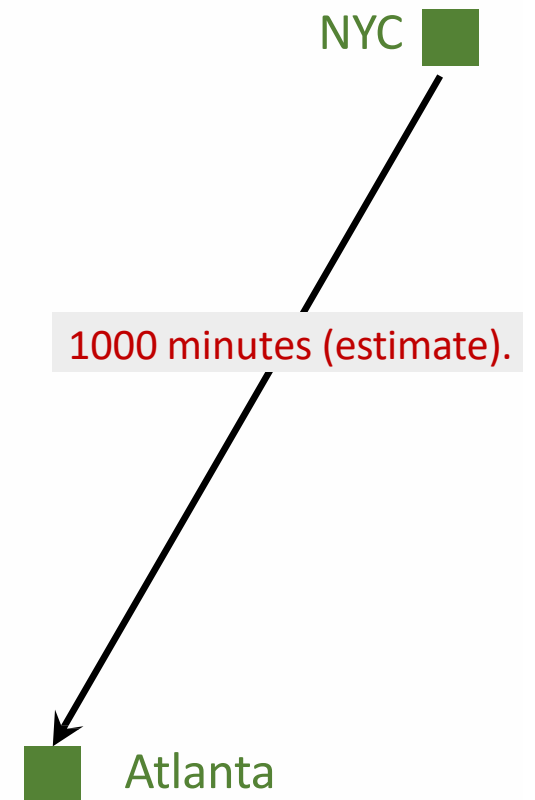
Question: How do I update the model?

- Can I update the model before finishing the trip?
- Can I get a better $\mathbf{w}$ as soon as I arrived at DC?



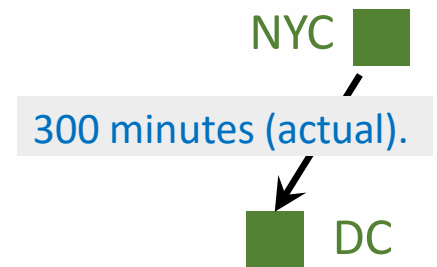
Temporal Difference (TD) Learning

- Model's estimate:
NYC to Atlanta: 1000 minutes (estimate).



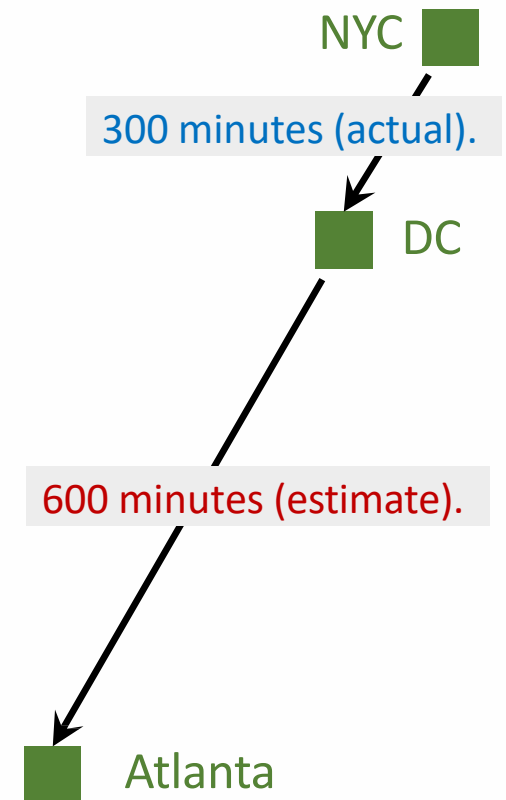
Temporal Difference (TD) Learning

- Model's estimate:
 NYC to Atlanta: 1000 minutes (estimate).
- I arrived at DC; actual time cost:
 NYC to DC: 300 minutes (actual).



Temporal Difference (TD) Learning

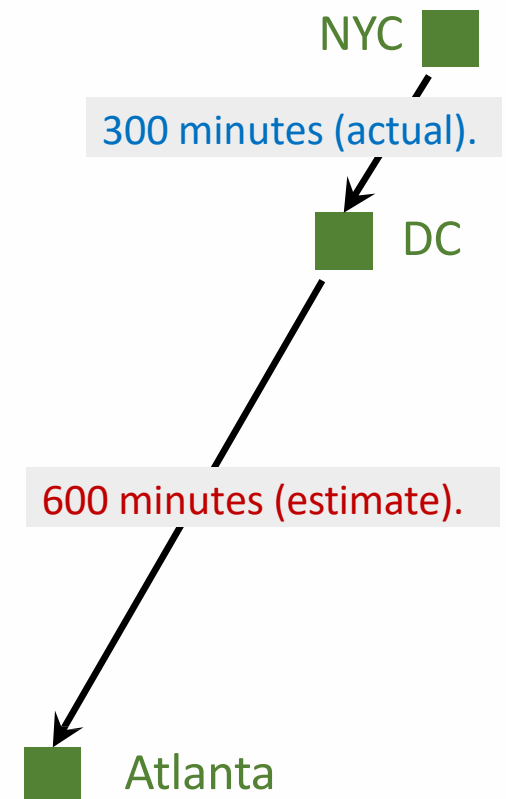
- Model's estimate:
 NYC to Atlanta: 1000 minutes (estimate).
- I arrived at DC; actual time cost:
 NYC to DC: 300 minutes (actual).
- Model now updates its estimate:
 DC to Atlanta: 600 minutes (estimate).



Temporal Difference (TD) Learning

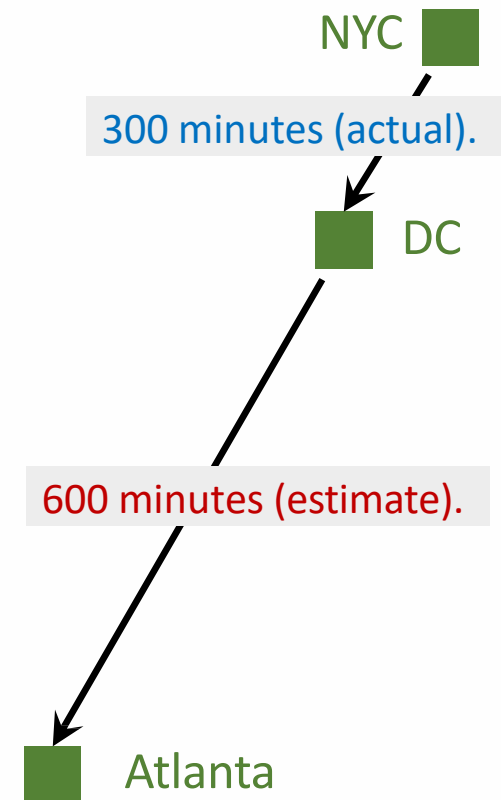
- Model's estimate: $Q(\mathbf{w}) = 1000$ minutes.
- Updated estimate: $300 + 600 = 900$ minutes.

TD target.



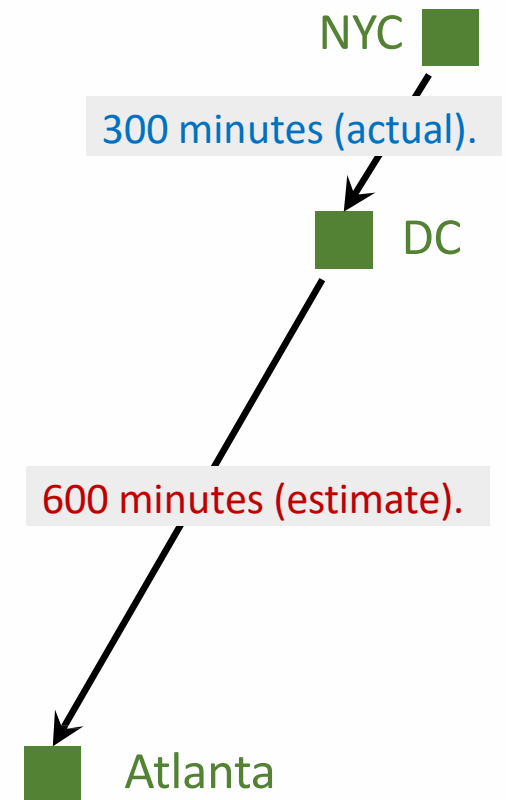
Temporal Difference (TD) Learning

- Model's estimate: $Q(\mathbf{w}) = 1000$ minutes.
- Updated estimate: $300 + 600 = 900$ minutes.
↓
TD target.
- TD target $y = 900$ is a more reliable estimate than 1000 .



Temporal Difference (TD) Learning

- Model's estimate: $Q(\mathbf{w}) = 1000$ minutes.
- Updated estimate: $300 + 600 = 900$ minutes.
TD target.
- TD target $y = 900$ is a more reliable estimate than 1000 .
- Loss: $L = \frac{1}{2} (\underbrace{Q(\mathbf{w}) - y}_{\text{TD error}})^2$.



Temporal Difference (TD) Learning

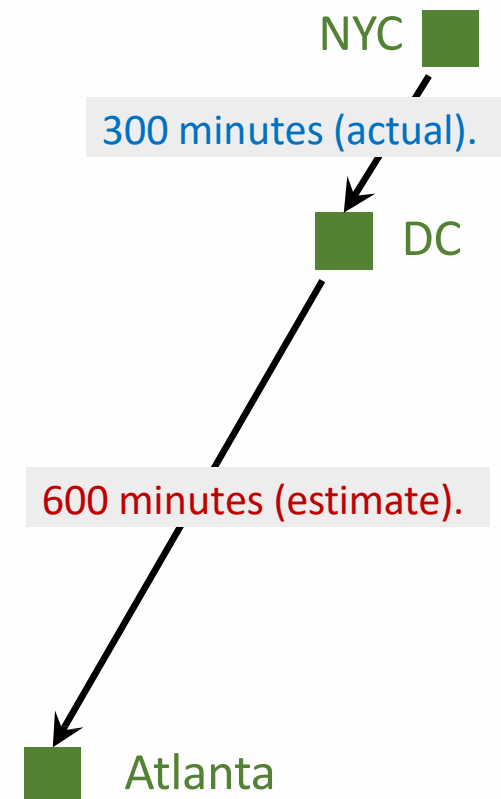
- Model's estimate: $Q(\mathbf{w}) = 1000$ minutes.
- Updated estimate: $300 + 600 = 900$ minutes.

↓
TD target.

- TD target $y = 900$ is a more reliable estimate than 1000 .

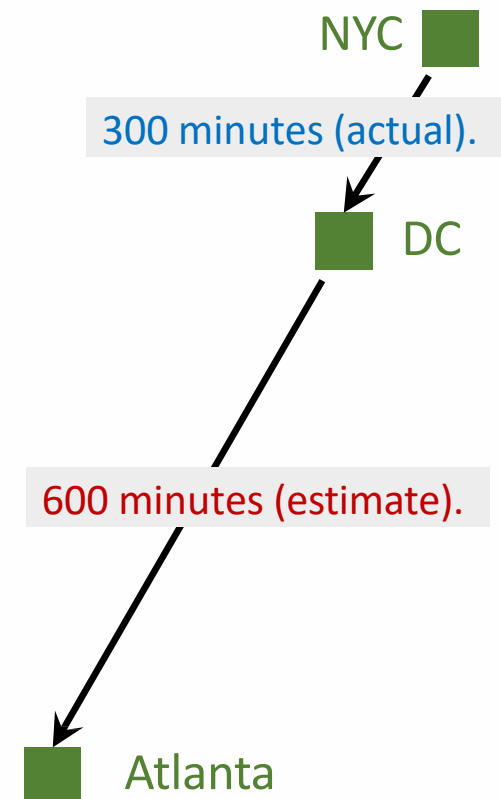
- Loss: $L = \frac{1}{2} (Q(\mathbf{w}) - y)^2$.

- Gradient: $\frac{\partial L}{\partial \mathbf{w}} = \underbrace{(1000 - 900)}_{\text{TD error}} \cdot \frac{\partial a(\mathbf{w})}{\partial \mathbf{w}}$.



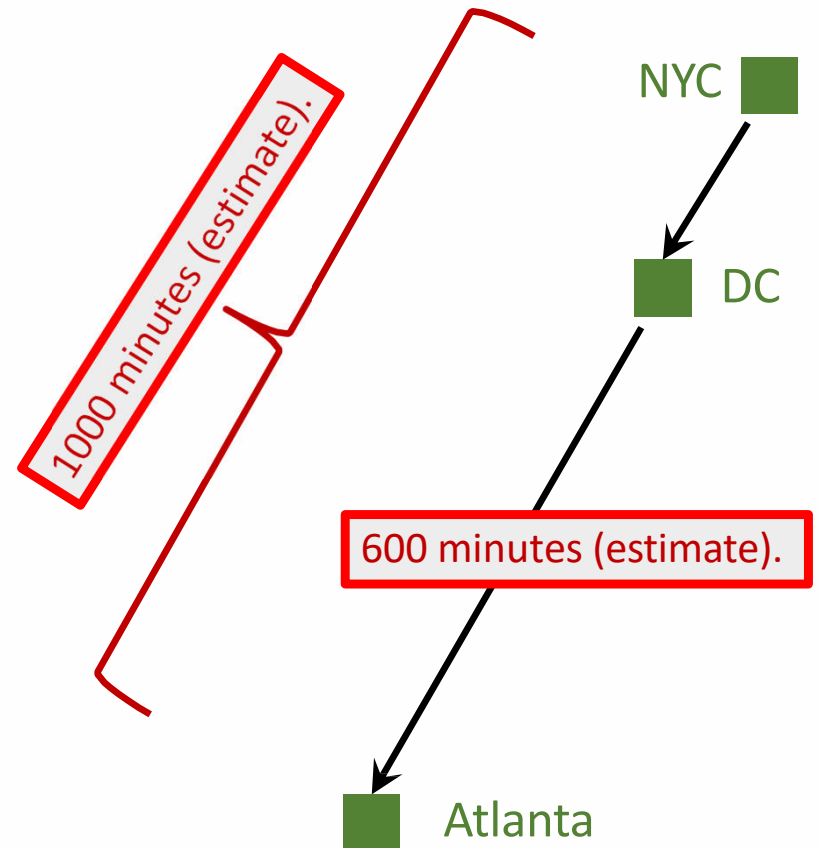
Temporal Difference (TD) Learning

- Model's estimate: $Q(\mathbf{w}) = 1000$ minutes.
- Updated estimate: $300 + 600 = 900$ minutes.
↓
TD target.
- TD target $y = 900$ is a more reliable estimate than 1000 .
- Loss: $L = \frac{1}{2} (Q(\mathbf{w}) - y)^2$.
- Gradient: $\frac{\partial L}{\partial \mathbf{w}} = (1000 - 900) \cdot \frac{\partial a(\mathbf{w})}{\partial \mathbf{w}}$.
- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{\partial L}{\partial \mathbf{w}} \big|_{\mathbf{w}=\mathbf{w}_t}$.



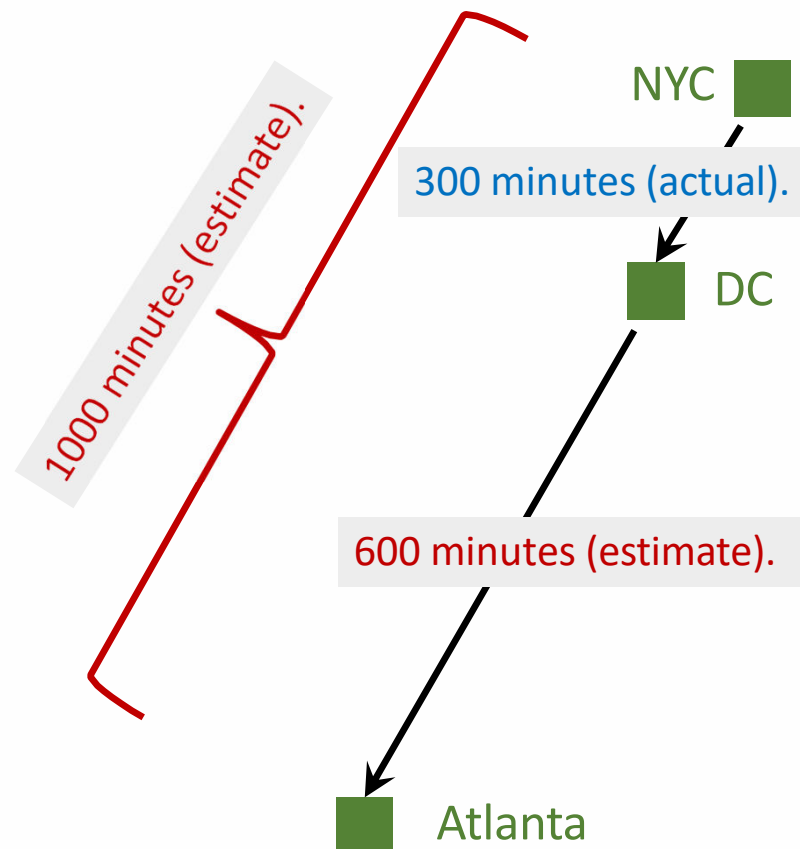
Why does TD learning work?

- Model's estimates:
 - NYC to Atlanta: 1000 minutes.
 - DC to Atlanta: 600 minutes.
 - ➔ NYC to DC: 400 minutes.



Why does TD learning work?

- Model's estimates:
 - NYC to Atlanta: 1000 minutes.
 - DC to Atlanta: 600 minutes.
 - → NYC to DC: 400 minutes.
- Ground truth:
 - NYC to DC: 300 minutes.
- TD error: $\delta = 400 - 300 = 100$



TD Learning for DQN

How to apply TD learning to DQN?

- In the “driving time” example, we have the equation:

$$T_{klm \rightarrow opq} \approx T_{klm \rightarrow rm} + T_{rm \rightarrow opq} .$$

The diagram illustrates the equation $T_{klm \rightarrow opq} \approx T_{klm \rightarrow rm} + T_{rm \rightarrow opq}$ with labels and arrows indicating the meaning of each term:

- $T_{klm \rightarrow opq}$ (red) is labeled "Model's estimate" (red) with a curved arrow pointing to it.
- $T_{klm \rightarrow rm}$ (blue) is labeled "Actual time" (blue) with a straight arrow pointing to it.
- $T_{rm \rightarrow opq}$ (red) is labeled "Model's estimate" (red) with a curved arrow pointing to it.

How to apply TD learning to DQN?

- In the “driving time” example, we have the equation:

$$T_{\text{NYC} \rightarrow \text{ATL}} \approx T_{\text{NYC} \rightarrow \text{DC}} + T_{\text{DC} \rightarrow \text{ATL}} .$$

Model's estimate Actual time Model's estimate

- In deep reinforcement learning:

$$Q(s_t, a_t; \mathbf{w}) \approx r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w}).$$

How to apply TD learning to DQN?

Definition of discounted return:

$$\begin{aligned} \bullet U_t &= R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \gamma^4 \cdot R_{t+4} + \dots \\ &\quad \underbrace{\hspace{10em}} \\ &= \gamma \cdot (R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \gamma^3 \cdot R_{t+4} + \dots) \end{aligned}$$

How to apply TD learning to DQN?

- $$\begin{aligned} U_t &= R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \gamma^4 \cdot R_{t+4} + \dots \\ &= R_t + \gamma \cdot \underbrace{(R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \gamma^3 \cdot R_{t+4} + \dots)}_{= U_{t+1}} \end{aligned}$$

How to apply TD learning to DQN?

Identity: $U_t = R_t + \gamma \cdot U_{t+1}$.

- $$\begin{aligned} U_t &= R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \gamma^4 \cdot R_{t+4} + \dots \\ &= R_t + \gamma \cdot \underbrace{(R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \gamma^3 \cdot R_{t+4} + \dots)}_{= U_{t+1}} \end{aligned}$$

How to apply TD learning to DQN?

Identity: $U_t = R_t + \gamma \cdot U_{t+1}$.

TD learning for DQN:

- DQN's output, $Q(s_t, a_t; \mathbf{w})$, is an estimate of U_t .
- DQN's output, $Q(s_{t+1}, a_{t+1}; \mathbf{w})$, is an estimate of U_{t+1} .

How to apply TD learning to DQN?

Identity: $U_t = R_t + \gamma \cdot U_{t+1}$.

TD learning for DQN:

- DQN's output, $Q(s_t, a_t; \mathbf{w})$, is an estimate of U_t .
- DQN's output, $Q(s_{t+1}, a_{t+1}; \mathbf{w})$, is an estimate of U_{t+1} .

• Thus,
$$\underbrace{Q(s_t, a_t; \mathbf{w})}_{\text{estimate of } U_t} \approx D[R_t] + \gamma \cdot \underbrace{Q(s_{t+1}, a_{t+1}; \mathbf{w})}_{\text{estimate of } U_{t+1}}.$$

How to apply TD learning to DQN?

Identity: $U_t = R_t + \gamma \cdot U_{t+1}$.

TD learning for DQN:

- DQN's output, $Q(s_t, a_t; \mathbf{w})$, is an estimate of U_t .
- DQN's output, $Q(s_{t+1}, a_{t+1}; \mathbf{w})$, is an estimate of U_{t+1} .

• Thus,
$$\underbrace{Q(s_t, a_t; \mathbf{w})}_{\text{Prediction}} \approx \underbrace{r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w})}_{\text{TD target}}.$$

Train DQN using TD learning

- Prediction: $Q(s_t, a_t; \mathbf{w}_t)$

- TD target:

$$\square y_t = r_t + \gamma \cdot Q(s_{t+1}, (a_{t+1}; \mathbf{w}_t)$$

Train DQN using TD learning

- Prediction: $Q(s_t, a_t; \mathbf{w}_t)$.
- TD target:

$$\begin{aligned} y_t &= r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w}_t) \\ &= r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}_t). \end{aligned}$$

Train DQN using TD learning

- Prediction: $Q(s_t, a_t; \mathbf{w}_t)$

- TD target:

$$\square y_t = r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w}_t)$$
$$= r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}_t).$$

- Loss: $L_t = \frac{1}{2} [Q(s_t, a_t; \mathbf{w}) - y_t]^2$.
- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{\partial L_t}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.

TD与Kalman滤波

- TD 的更新形式: $\leftarrow$

$$V \leftarrow V + \alpha \cdot (\text{目标} - \text{当前估计}) \leftarrow$$

$\underbrace{\hspace{1.5cm}}$
TD error

- Kalman 更新式: $\leftarrow$

$$\text{新估计} = \text{旧预测} + K_t \cdot (\text{观测} - \text{旧预测}) \leftarrow$$

$\underbrace{\hspace{1.5cm}}$
residual

TD 像 Kalman 的 update 形式: $\leftarrow$

- TD 步长 $\alpha \approx$ Kalman 增益 K_t (一个固定/自适应的权重)
- TD 误差 $\delta \approx$ Kalman 的“观测残差” $(z - H\hat{x}) \leftarrow$

Summary

Value-Based Reinforcement Learning

Definition: Optimal action-value function.

- $Q^*(s_t, a_t) = \max_{\pi} \mathbb{D}[U_t | S_t = s_t, A_t = a_t]$.

Value-Based Reinforcement Learning

Definition: Optimal action-value function.

- $Q^*(s_t, a_t) = \max_{\pi} \mathbb{E} [U_t | S_t = s_t, A_t = a_t]$.

DQN: Approximate $Q^*(s, a)$ using a neural network (DQN).

- $Q(s, a; \mathbf{w})$ is a neural network parameterized by $\mathbf{w}$.
- Input: observed state s .
- Output: scores for all the action $a \in \mathcal{A}$.

Temporal Difference (TD) Learning

Algorithm: One iteration of TD learning.

1. Observe state $S_t = s_t$ and perform action $A_t = a_t$.
2. Predict the value: $q_t = Q(s_t, a_t; \mathbf{w}_t)$.
3. Differentiate the value network: $\mathbf{d}_t = \frac{\partial a(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.

Temporal Difference (TD) Learning

Algorithm: One iteration of TD learning.

1. Observe state $S_t = s_t$ and perform action $A_t = a_t$.
2. Predict the value: $q_t = Q(s_t, a_t; \mathbf{w}_t)$.
3. Differentiate the value network: $\mathbf{d}_t = \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.
4. Environment provides new state s_{t+1} and reward r_t .
5. Compute TD target: $y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}_t)$.

Temporal Difference (TD) Learning

Algorithm: One iteration of TD learning.

1. Observe state $S_t = s_t$ and perform action $A_t = a_t$.
2. Predict the value: $q_t = Q(s_t, a_t; \mathbf{w}_t)$.
3. Differentiate the value network: $\mathbf{d}_t = \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.
4. Environment provides new state s_{t+1} and reward r_t .
5. Compute TD target: $y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}_t)$.
6. Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot (q_t - y_t) \cdot \mathbf{d}_t$.

